

**B** Academy  
**RO**



**Manual**

**Flex and position**

# document flow

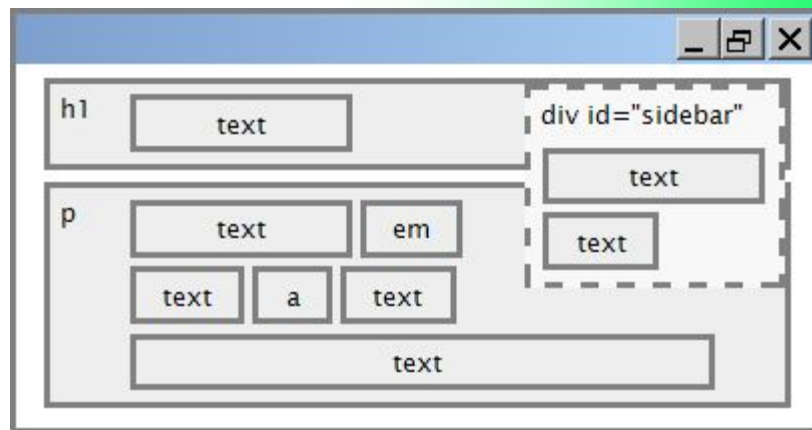
– is an abstract model of how elements on a webpage are arranged and interact with each other. It determines the order in which elements will be displayed on the page.

Browsers use special rules to define how items are displayed on the screen, which makes this possible.

- The standard flow runs from left to right, top to bottom. The flow is maintained in any element.
- The flow consists of two types of elements:
  - **inline**
  - **block**

# Properties Affecting Document Flow

- display
- width, height
- position
- float
- column-width, column-count
- gap, column-gap, row-gap
- margin, padding, border



# Content

**block-level boxes** – Occupy the entire available horizontal width and start on a new line

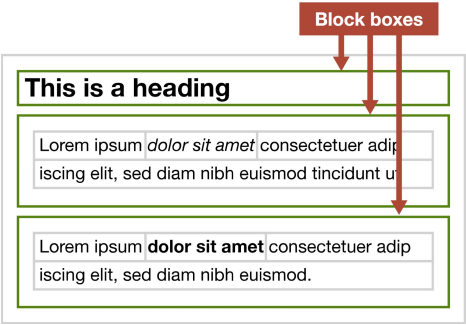
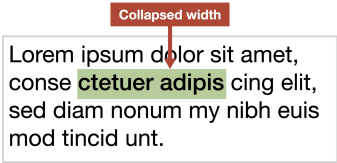
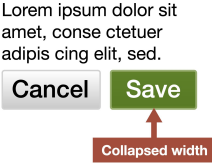
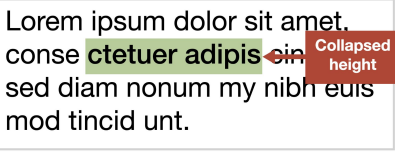
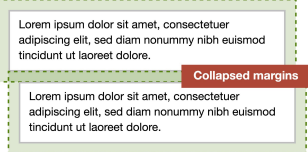
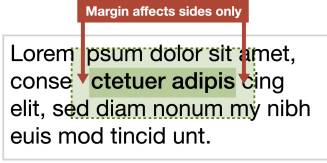
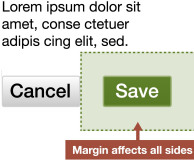
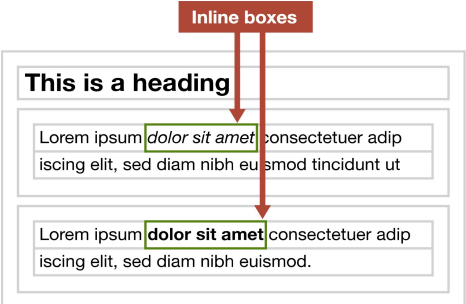
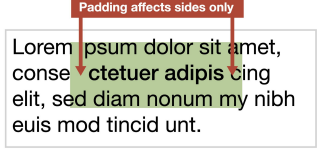
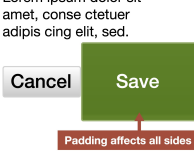
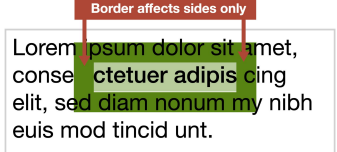
`display: block, list-item, table, flex, grid`

|                                 |                                 |                                    |                         |                            |                              |
|---------------------------------|---------------------------------|------------------------------------|-------------------------|----------------------------|------------------------------|
| <code>&lt;blockquote&gt;</code> | <code>&lt;fieldset&gt;</code>   | <code>&lt;h1&gt;-&lt;h6&gt;</code> | <code>&lt;ol&gt;</code> | <code>&lt;div&gt;</code>   | <code>&lt;section&gt;</code> |
| <code>&lt;dd&gt;</code>         | <code>&lt;figcaption&gt;</code> | <code>&lt;header&gt;</code>        | <code>&lt;li&gt;</code> | <code>&lt;p&gt;</code>     | <code>&lt;main&gt;</code>    |
| <code>&lt;dl&gt;</code>         | <code>&lt;figure&gt;</code>     | <code>&lt;footer&gt;</code>        | <code>&lt;ul&gt;</code> | <code>&lt;table&gt;</code> | <code>&lt;article&gt;</code> |
| <code>&lt;dt&gt;</code>         | <code>&lt;form&gt;</code>       | <code>&lt;aside&gt;</code>         | <code>&lt;hr&gt;</code> | <code>&lt;nav&gt;</code>   |                              |

**inline-level boxes** – Occupy only the necessary horizontal space and are placed on the same line. They don't start on a new line and take up only as much space as they need.

`display: inline, inline-block, inline-table, inline-flex, inline-grid`

|                             |                           |                         |                               |                             |                           |
|-----------------------------|---------------------------|-------------------------|-------------------------------|-----------------------------|---------------------------|
| <code>&lt;a&gt;</code>      | <code>&lt;time&gt;</code> | <code>&lt;i&gt;</code>  | <code>&lt;label&gt;</code>    | <code>&lt;strong&gt;</code> | <code>&lt;cite&gt;</code> |
| <code>&lt;span&gt;</code>   | <code>&lt;kbd&gt;</code>  | <code>&lt;q&gt;</code>  | <code>&lt;textarea&gt;</code> | <code>&lt;sub&gt;</code>    | <code>&lt;abbr&gt;</code> |
| <code>&lt;button&gt;</code> | <code>&lt;code&gt;</code> | <code>&lt;b&gt;</code>  | <code>&lt;select&gt;</code>   | <code>&lt;sup&gt;</code>    |                           |
| <code>&lt;img&gt;</code>    | <code>&lt;br&gt;</code>   | <code>&lt;em&gt;</code> | <code>&lt;input&gt;</code>    |                             |                           |

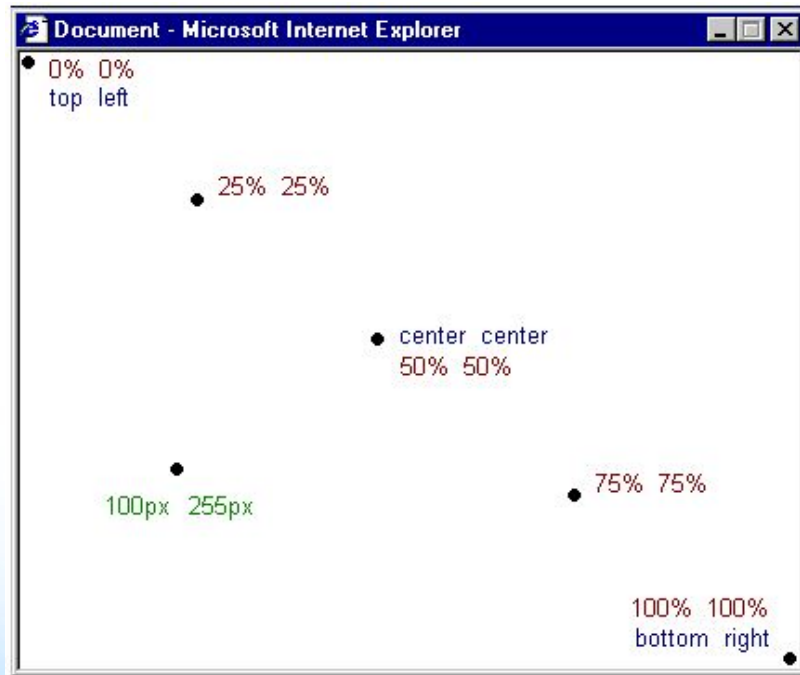
|                | block boxes                                                                       | inline boxes                                                                        | inline-block boxes                                                                   |
|----------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Default width  |  |   |   |
| Default height |                                                                                   |   |   |
| Margins        |  |   |   |
| Padding        |  |   |   |
| Border         |                                                                                   |  |  |

The **position** property specifies the type of positioning method used for an element:

- static
  - relative
  - absolute
  - fixed
  - sticky
- 
- Positioning should not be used for creating layouts, only for decorative purposes

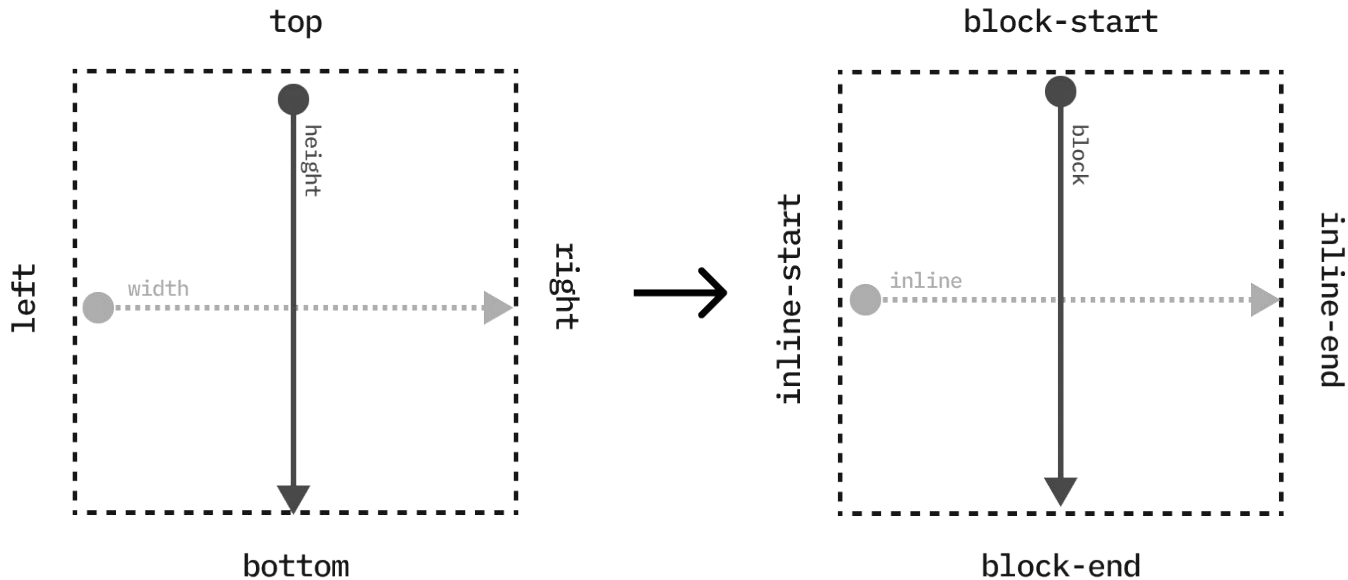
# CSS POSITION

- **Do not use margin** to set the position of such an element.
- Units of measurement:
  - px
  - %
  - **top, right, bottom, left** (can be negative.)
  - **inset** – shorthand property for **top, right, bottom, left**
- should **always** have at least **one horizontal and one vertical** coordinate specified
- You can specify all four coordinates simultaneously, but there's no guarantee that they will all work together as expected.



# CSS COORDINATE SYSTEM

# logical properties coordinates





position:



- This is the default value for all elements.
- Elements take their place in the flow depending on the box type and the amount of content.
- `top`, `right`, `bottom`, `left`, `z-index` don't work
- **normal document flow**



`position: static;`

position:

**relative** 

- The element is **removed from the normal document flow**.
- It is positioned relative to its **original position in the flow**.
- Space for the element is **still reserved** in the document flow.
- The height is reset to **height: auto**
- normal document flow



```
position: relative;  
left: 20px;  
top: 20px;
```

position:

**absolute** 

- The element is **removed from the document flow**, and neighboring elements completely ignore it.
- It is positioned **relative to its parent block** or the nearest ancestor with **position: relative**.
- The width defaults to **width: auto**, making it as wide as its content  
**Padding and border** are added to this width.
- If both horizontal or both vertical coordinates are specified, the element will stretch between them.



```
position: absolute;  
left: 0;  
top: 10px;
```

## position:



- The element is removed from the document flow, and neighboring elements completely ignore it.
- It is positioned relative to the **viewport**.
- It does not move when **the page is scrolled**.
- The width defaults to **width: auto**, and the height to **height: auto**, making it sized according to its content. Note: **padding**, **border** are added to these dimensions.
- If both horizontal or both vertical coordinates are specified, the element will stretch between them.



```
position: fixed;  
left: 20px;  
top: 20px;
```

position:

**sticky** 

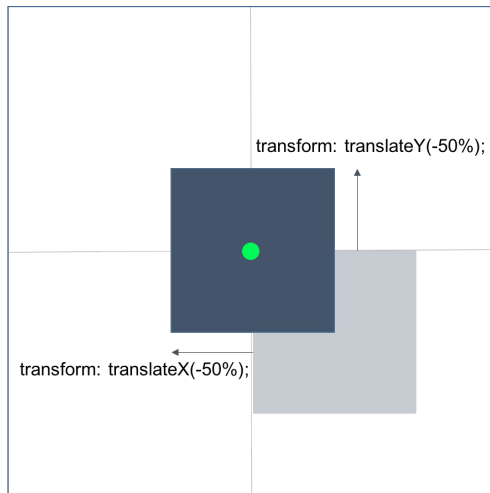
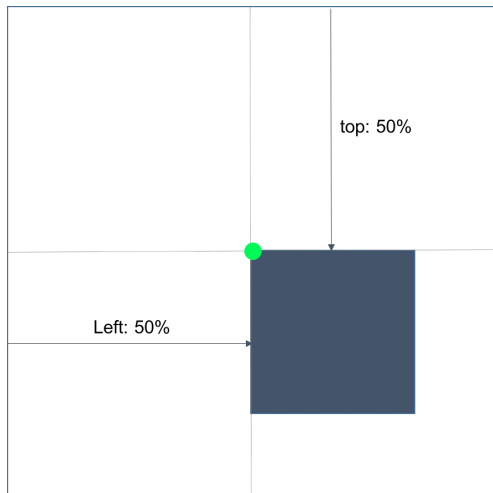
- The element is removed from the document flow.
- Neighboring elements behave as if the element is still in its original position, and space for the element is still reserved in the document flow.
- The element does not move when its containing **block is scrolled**.
- It behaves like **position: relative** until the block is scrolled, after which it behaves like **position: fixed**
- The behavior of **padding, border, margin** depends on the box type—whether it is **block-level** or **inline-level**.



```
position: sticky;  
right: 0;  
top: 10px;
```

# center position

## Demo



Adds an offset upward by half the height of the block using:  
**transform: translateY(-50%);**

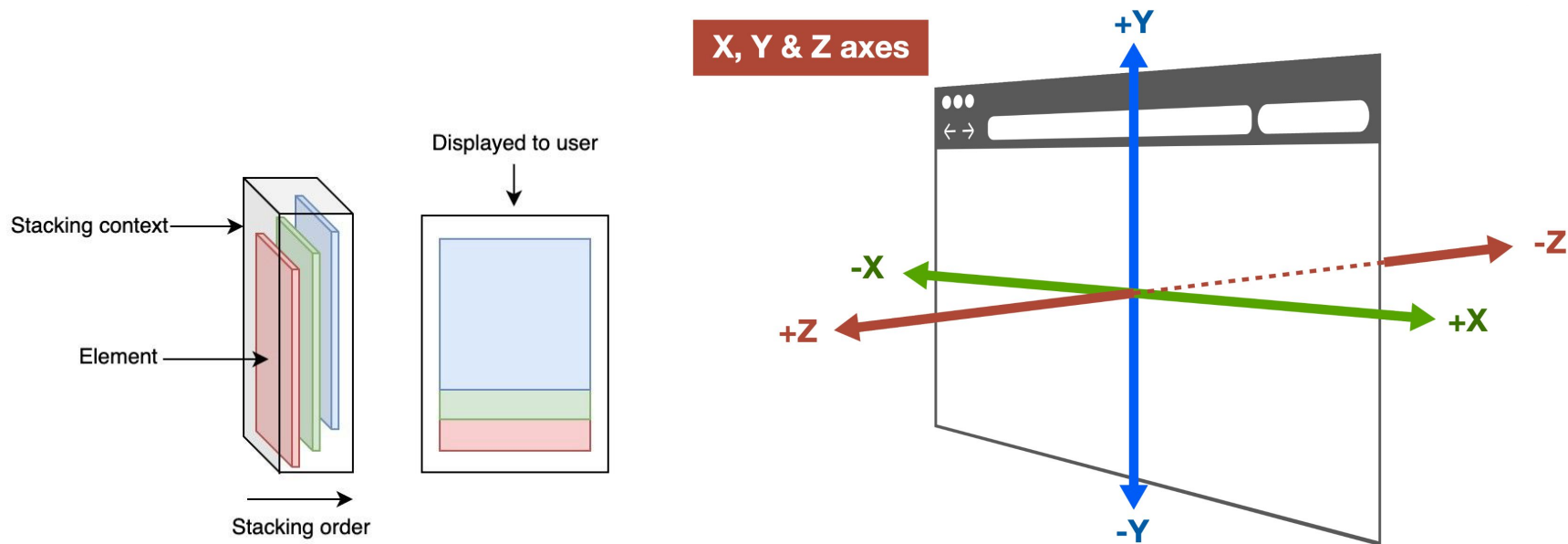
Adds an offset to the left by half the width of the block using:  
**transform: translateX(-50%);**

or

**translate: -50% -50%;**

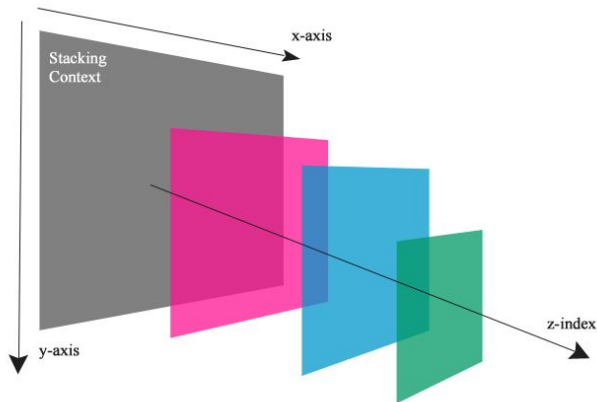
# stacking context

– is a three-dimensional conceptualization of HTML elements along an imaginary z-axis relative to the user, who is assumed to be facing the viewport or the webpage. HTML elements occupy this space in priority order based on element attributes.



# z-index

Demo



- Controls the stacking order of elements within a stacking context.
- Applies only to elements with an explicitly defined `position`
- If positioning is not needed but you still require a z-index, use `position:relative` without coordinates.
- Values can be positive or negative numbers.
- The higher the number, the more visible the layer.
- By default, `z-index: auto` is used, and visibility is determined by the element's position in the DOM tree: the lower it is in the code, the more visible it is.



# stacking context

**root stacking context** – `<html>` tag creates the root stacking context, unless other conditions are specified:

- **position: absolute** with a **z-index** other than **auto**
- **position: relative** with a **z-index** other than **auto**
- **position: fixed**
- **position: sticky** (only on mobile devices)
- A child element of a **flex** container with a **z-index** other than **auto**
- **opacity** less than **1**
- **transform** with a value other than **none**
- **filter** with a value other than **none**
- **perspective** with a value other than **none** or **0**

# float

allows content to flow around a block element.

When an element is floated, it is removed from the normal document flow and pushed as far as possible to the specified side (left or right) within its containing element.

**float: left;**  
**: right;**  
**: both;**  
**: none;**

**Do not use float for building layouts!**

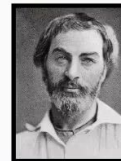


Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor in

cidunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor in cididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.

“This is what you shall do; ~~Love~~ the earth and sun and the animals, despise riches, give alms to every one that asks, stand up for the stupid and crazy, devote your income and labor to others, hate tyrants, argue not concerning God, have patience and indulgence toward the people, take off your hat to nothing known or unknown or to any man or number of men, go freely with powerful uneducated persons and with the young and with the mothers of families, read these leaves in the



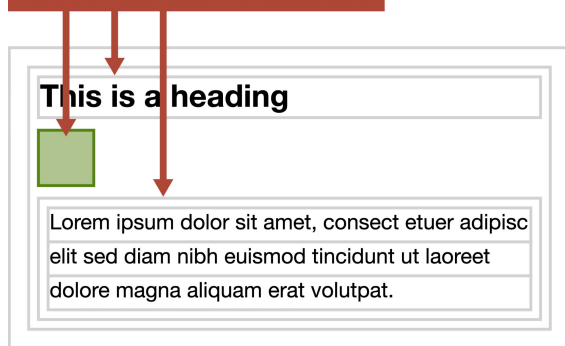
Walt Whitman



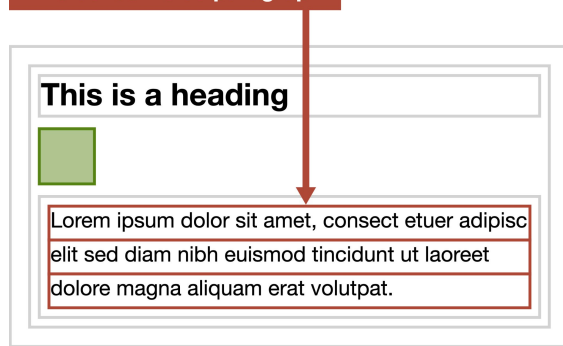
Photo by Jeremy Bishop on Unsplash

open air every season of every year of your life, re-examine all you have been told at school or church or in any book, dismiss whatever insults your own soul, and your very flesh shall be a great poem and have the richest fluency not only in its words but in the silent lines of its lips and face and between the lashes of your eyes and in every motion and joint of your body.” – Walt Whitman, *Song of Myself*

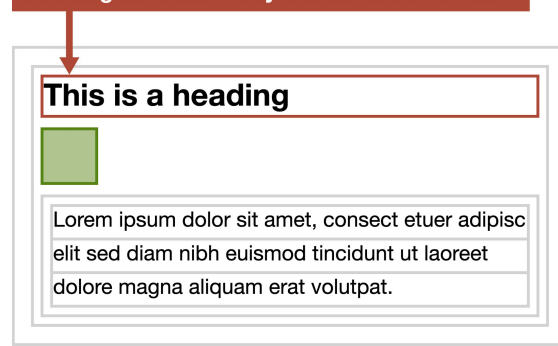
All three elements in normal flow



Line boxes inside paragraph

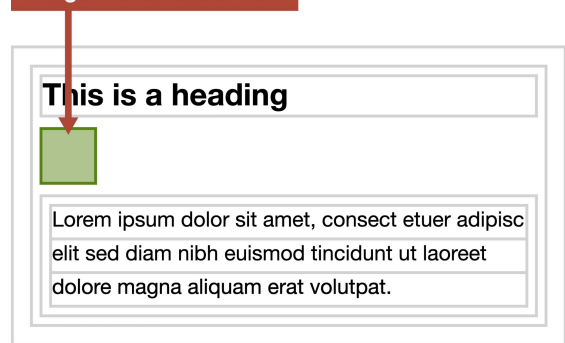


Heading not affected by floated element below



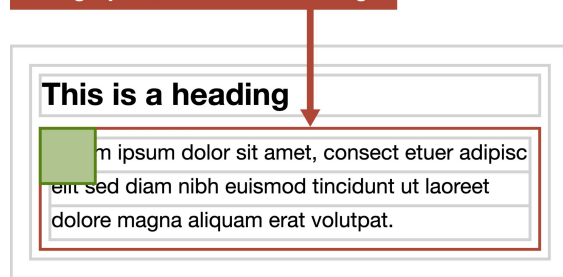
1

Image moves out of flow



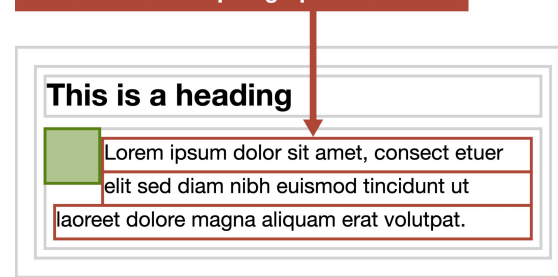
2

Paragraph slides under the image



3

Line boxes inside paragraph will shorten



# **clear** float

is used to stop elements from being wrapped by floated elements.

It ensures that the element to which it is applied does not sit next to or wrap around any floated.

When you have an element that you want to appear beneath floated elements (rather than beside them), you apply **clear** to that element.

The most common scenario is applying clear to the element that follows a floated element to ensure that it does not wrap around the float but instead starts on a new line below it.

```
clear: left;  
       : right;  
       : both;  
       : none;
```

**Demo**

It can be applied to:

## 1. An element that follows a floated element - **float**

The floating element is floated to the left, which causes the content to wrap around it.

The current element is floated to the right but has the `clear: left;` property applied. This causes the current element to drop down below the left-floated element, preventing any content from being next to it on the left side. However, since it is also floated right, it aligns itself to the right edge after moving down.

The floating element is again floated to the left, causing content to wrap around it.

The current element is also floated to the left and has `clear: right;` applied. However, because `clear: right;` doesn't affect left-floated elements, the current element aligns itself to the left directly next to the first element, maintaining the float on the same side. The `clear: right;` has no visible effect in this scenario because there's no element floated to the right.

### FLOATING-ELEMENT

`{float: left}`



Lorem ipsum dolor sit amet, con sectetur adip iscing elit, sed do eius mod tempor in cididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exer citat ion ullamco laboris nisi ut aliquip ex ea com modo consequat. Duis aute irure dolor in repre henderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.

Lorem ipsum dolor sit amet, con sectetur adip iscing elit, sed do eius mod tempor in cididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exer citat ion ullamco laboris nisi ut aliquip ex ea com modo consequat. Duis aute irure dolor in repre henderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.



### CURRENT-ELEMENT

`{float: right}`

`{clear: left}`

### FLOATING-ELEMENT

`{float: left}`

### CURRENT-ELEMENT

`{float: left}`

`{clear: right}`



Lorem ipsum dolor sit amet, con sectetur adip iscing elit, sed do eius mod tempor in cididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exer citat ion ullamco laboris nisi ut aliquip ex ea com modo consequat. Duis aute irure dolor in repre henderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.

>Lorem ipsum dolor sit amet, con sectetur adip iscing elit, sed do eius mod tempor in cididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exer citat ion ullamco laboris nisi ut aliquip ex ea com modo consequat. Duis aute irure dolor in repre henderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.

>Lorem ipsum dolor sit amet, con sectetur adip iscing elit, sed do eius mod tempor in cididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exer citat ion ullamco laboris nisi ut aliquip ex ea com modo consequat. Duis aute irure dolor in repre henderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.

It can be applied to:

1. An element that follows a floated element - **float**

In this scenario, the floating element is floated to the left.

The current element is also floated to the left but has the `clear: left` property applied. This causes the current element to drop down below the left-floated element, clearing the float on the left side. The text and other content no longer wrap around the floating element, and the current element starts on a new line directly below the first floating element.

#### FLOATING-ELEMENT

`{float: left}`

#### CURRENT-ELEMENT

`{float: left}`

`{clear: left}`



Lorem ipsum dolor sit amet, con sectetur adip iscing elit, sed do eius mod tempor in cididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exer citat ion ullamco laboris nisi ut aliquip ex ea com modo consequat. Duis aute irure dolor in repre henderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.

im ad minim veniam, quis nostrud exer citat ion ullamco laboris nisi ut aliquip ex ea com modo consequat. Duis aute irure dolor in repre henderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.

It can be applied to:

## 2. An element that contains a floated element

In this example, the current element has `clear: left`; applied. This ensures that the current element does not allow any left-floating elements to sit beside it, so it starts on a new line.

The floating element is floated to the right, causing the content to wrap around it on the left side.

Since there is no left-floating element before the current element, the `clear: left`; rule has no effect on the floating element positioned on the right side.

Here, the current element has `clear: right`; applied, which ensures that no elements floated to the right can be next to it. The current element is forced to the next line beneath the right-floating element.

The floating element is also floated to the right, causing content to wrap around it on the left. But because of the `clear: right`; property on the current element, it is pushed below the floated element and does not allow any content to be next to it on the right.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor in



cididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor in cididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.

**CURRENT-ELEMENT**

`{clear: left}`

**FLOATING-ELEMENT**

`{float: right}`



Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor in cididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor in cididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.

**CURRENT-ELEMENT**

`{clear: right}`

**FLOATING-ELEMENT**

`{float: right}`

# clear float

## Alternatives:

**overflow:** `hidden;` 

This technique is often used to contain floated elements within a parent element, clearing the floats without the need for additional markup or clearing elements.

**display:** `flow-root;` 

A modern CSS property that establishes a new block formatting context, effectively containing floats and clearing them automatically.